Multi – level Linked List

–Modified Insert After Element

–Space Complexity and Algorithm

Program:

```
Node *searchNode(Node *current, int target)
{
    if (current == nullptr)
        return nullptr;

    if (current->data == target)
        return current;

    Node *foundInChild = searchNode(current->child, target);
    if (foundInChild != nullptr)
    {
        return foundInChild;
    }
    return searchNode(current->next, target);
}

void insertAfterElement()
{
    if (head == nullptr)
    {
        cout << "List is empty. Please insert an element
first." << endl;
```

```
        return;
    }

    int target, data, type;
    cout << "Enter the element to search for: ";
    cin >> target;

    Node *temp = searchNode(head, target);

    if (temp == nullptr)
    {
        cout << "Element " << target << " not found anywhere
in the list." << endl;
        return;
    }

    cout << "Enter data to insert: ";
    cin >> data;
    Node *newNode = createNode(data);

    cout << "Insert as (1) Next Node or (2) Child Node? ";
    cin >> type;

    if (type == 1)
    {
        newNode->next = temp->next;
        temp->next = newNode;
```

```
        cout << "Inserted " << data << " as NEXT of " <<
target << endl;
    }
    else if (type == 2)
    {
        newNode->next = temp->child;
        temp->child = newNode;
        cout << "Inserted " << data << " as CHILD of " <<
target << endl;
    }
    else
    {
        cout << "Invalid option." << endl;
        free(newNode);
    }
}
int main()
{
    int choice;
    while (true)
    {
        cout << "4. Insert After Element (Link or Child)" <<
endl;
       switch (choice)
        {
       case 4:
            insertAfterElement();
        default:
```

```
            cout << "Invalid choice" << endl;
        }
    }
    return 0;
}
```

Space Complexity – Analysis

1. *Input Space Complexity*

For Input Space Complexity, we consider only the memory occupied by the input, not the temporary variables or recursion stack (those belong to auxiliary space).

Let the multilevel linked list contain N nodes.

Each node is:

```
struct Node
{
    int data;
    struct Node *next;  // Points to the right (Same Level)
    struct Node *child; // Points down (Sub Level)
};
```

A. Parameter-Based Input Space

Here we consider only the parameters passed to the functions.

---

1. searchNode()

```
Node *searchNode(Node *current, int target)
```

Parameters are:

- $Node * current$
- $int\ target$

Memory used

Parameter	*Size*
$Node * current$	1 *pointer*
$int\ target$	1 *integer*

Total

$$1 \text{ pointer} + 1 \text{ integer} = O(1)$$

Therefore,

$$\boxed{\text{Parameter-Based Input Space of searchNode()} = O(1)}$$

B. Full Data / Holistic Input Space

Here we consider all data supplied to the algorithm.

The program operates on an existing multilevel linked list.

- $Number\ of\ nodes\ =\ N$

Each node stores:

- ***one integer*** $(\boldsymbol{data})$
- ***two pointers***$(\boldsymbol{next}, \boldsymbol{child})$

So every node occupies constant memory.

Hence total input memory is:

$$\boldsymbol{N} \times \boldsymbol{O}(\mathbf{1})$$

Therefore,

$$\boxed{\boldsymbol{O}(\boldsymbol{N})}$$

Auxiliary Space:

The only significant extra memory used by this program comes from the recursive function.

```
Node *searchNode(Node *current, int target)
```

because every recursive call creates one activation record (stack frame).

Each stack frame stores only constant − sized information:

- ***parameter*** $Node * current$
- ***parameter*** $int\ target$
- ***local pointer*** $Node * foundInChild$
- ***return address***
- ***saved registers/bookkeeping***

Hence,

$$Each\ recursive\ call\ uses\ O(1)\ space.$$

Therefore,

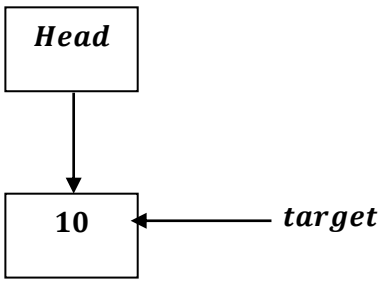
$$Auxiliary\ Space = (maximum\ recursion\ depth) \times O(1)$$

The recursion depth depends entirely on the shape of the multilevel linked list.

---

1) *Best Case*

Suppose



The first node matches.

Execution:

$\boldsymbol{searchNode(head, target) \rightarrow found\ immediately\ and\ return.}$

Only one stack frame exists.

Therefore,

$$\boldsymbol{S(N) = c}$$

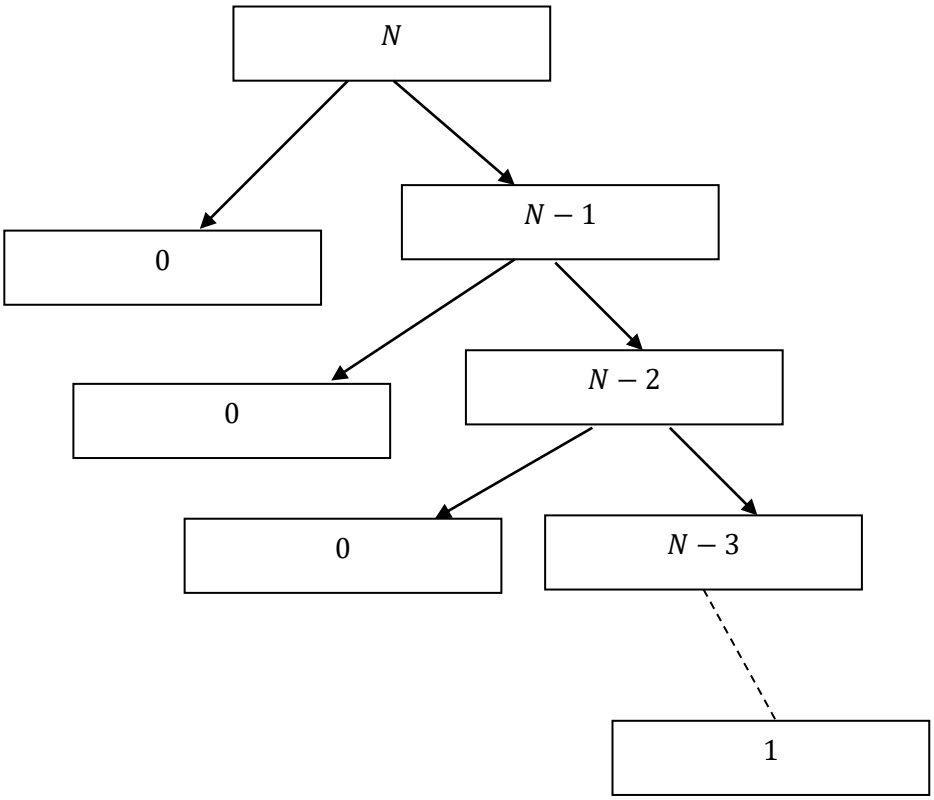
Hence

$$\boldsymbol{O(1)}$$

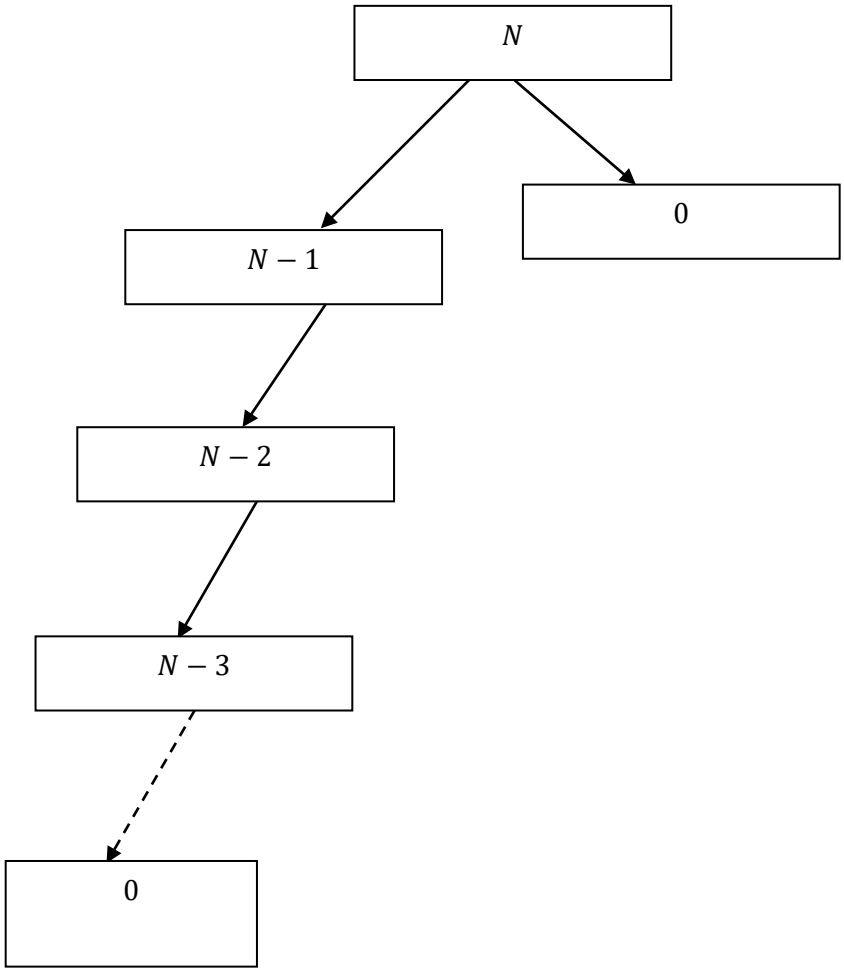
---

2) *Worst Case*

Right Skewed



Left Skewed



$\boldsymbol{Level}$ (k)	$\boldsymbol{No.of\ problems}$	$\boldsymbol{Problem\ Size}$	$\boldsymbol{Work\ /Cost\ Per\ Problem}$	$\boldsymbol{Work\ done = Work\ per\ Problem \times No.of\ Problems}$
$\mathbf{0}$	$\mathbf{1}$	$\boldsymbol{n-0=n}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
$\mathbf{1}$	$\mathbf{1}$	$\boldsymbol{n-1}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
$\mathbf{2}$	$\mathbf{1}$	$\boldsymbol{n-2}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
. .	. .	. .	. .	. .
$\boldsymbol{n-1}$	$\mathbf{1}$	$\boldsymbol{n-(n-1)=1}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
$\boldsymbol{n}$ $(\boldsymbol{Base\ Case})$	$\mathbf{1}$	$\boldsymbol{n-n=0}$ $(\boldsymbol{Base\ Case})$	$\boldsymbol{T(0)}$	$\boldsymbol{1\times T(0)=T(0)}$

$\boldsymbol{Last\ level = N\ , hence\ , maximum\ depth = N.}$

$\boldsymbol{Hence}$

$$\boldsymbol{O(N)}$$

---

3) $Worst\ \ Case$

$$2T\left(\frac{N-1}{2}\right)+C\ ,Base\ Case: T(0)=1.$$

$Level\ (k)$	$No.of\ problems$	$Problem\ Size$	$Work\ /Cost\ Per\ Problem\ (Print\ and\ free\ memory)$	$Work\ done = Work\ per\ Problem \times No.of\ Problems$
0	1	n	C	$1\times C=C$
1	2	$\frac{n-1}{2}$	C	$2\times C=2C$
2	4	$\frac{n-3}{4}$	C	$4\times C=4C$
3	8	$\frac{n-7}{8}$	C	$8\times C=8C$
. .	. .	. .	. .	. .
k	2^k	$\frac{N-(2^k-1)}{2^k}$	C	$2^k\times C=2^kC$
. . .	. . .	. . .	. . .	. . .
$\log_2(N+1)$ $(Base\ Case)$	$N+1$	$\frac{N-(2^{\log_2(N+1)}-1)}{2^{\log_2(N+1)}}=0$ $(Base\ Case)$	$T(0)$	$(N+1)\times T(0)$

$Last\ level=\ \log_2(N+1), hence, maximum\ depth=\log_2(N+1).$

$As\ , Base\ Case=T(0), if\ T(1) then\ , \log_2(N+1)-1.$

$$\textbf{\textit{Here } } T(0), \textbf{\textit{so Last level}} = \log_2(N+1), \textbf{\textit{hence, maximum depth}}$$
$$= \log_2(N+1).$$

$$= O\left(\log_2(N+1)\right)$$
$$= O(\log_2 N)$$

Algorithm :

Function: ***insertAfterElement***():

1. ***if START = NULL, Then***:

   1. a. ***Write***: ``***LIST IS EMPTY. PLEASE INSERT AN ELEMENT FIRST.***``

   1. b. ***Return void and exit.***

[***END IF***]

2. ***Declare three integer type variable target, data, type.***

3. ***Write***: ``***Enter the element to search for***``

4. ***Read target.***

5. ***SET TEMPPTR*** := ***searchNode***(***START***, ***TARGET***)

— —

SET CURRPTR := ***START***

SET TARGET := ***TARGET***

Translating searchNode(***CURRPTR***, ***TARGET***):

if CURRPTR = NULL , THEN:

RETURN NULL

[***END of IF***]

if INFO[***CURRPTR***] = ***TARGET , THEN***:

RETURN CURRPTR

[***END of IF***]

$SET\ FOUNDInCHILDPTR := \ searchNode(CHILD[CURRPTR], TARGET)$

$if\ \ FOUNDInCHILDPTR \neq NULL\ , THEN:$

$RETURN\ FOUNDInCHILDPTR$

$[END\ of\ IF]$

$RETURN\ searchNode(LINK[CURRPTR], TARGET)$

$-----------------------------$

$6. [If\ Out\ of\ Bound\ ?]$

$If\ TEMPPTR = NULL, THEN:$

$Write: ``Element``, TARGET, ``\ not\ found$

$in\ the\ main\ list.``$

$RETURN\ void\ and\ exit.$

$[End\ of\ If\ structure.]$

$7. Write: ``Enter\ data\ to\ insert: ``$

$8. Read\ data.$

$9. SET\ NEWPTR := CALL\ CreateNode(\ data)$

$Translating\ CreateNode(data)$

$(((Node\ *)malloc(sizeof(Node))\ is\ availability\ of\ space\ for\ insertion$

$of\ Data, hence, lets\ name\ it\ AVAIL.))$

$CreateNode(data)$:

1. $Allocate\ New\ Node$

   $SET\ \ NewTempPTR := AVAIL.$

2. $Memory\ Allocated$?:

   $If\ NewTempPTR\ =\ NULL, then$:
   $Write$: " Error: Memory allocation failed."
   $Return\ and\ EXIT.$
   $[End\ of\ If\ structure.]$

3. $Copy\ Data$:

   $Set\ INFO[NewTempPTR]\ := data$

4. $Set\ Next\ section\ of\ Allocated\ Memory$:

   $Set\ Link[NewTempPTR]\ := NULL$

5. $Set\ Child\ section\ of\ Allocated\ Memory$:

   $Set\ childLink[NewTempPTR]\ := NULL$

6. $Return\ Stored\ Address\ in\ NewTempPTR$

   $return\ NewTempPTR$

---

10. ***Write***: ``***Insert as*** **(1)** ***Next Node or*** **(2)** ***Child Node***?``

11. ***Read type***.

12. [***Insertion occurs in Main List***]

if type $=$ **1** , ***then*** :

SET LINK[***NEWPTR***] $:=$ ***LINK***[***TEMPPTR***]

[*LINKING NEWNODE with the previous node after which newNode is to be placed in LINK LIST*]

SET LINK[***TEMPPTR***] $:=$ ***NEWPTR***

[*LINKING NEWNODE with the NEXT NODE where this NEW NODE is to be placed in LINK LIST*]

Write : `` ***Inserted***`` , ***data*** , ``***as NEXT of*** ``, ***target*** .

[***Insertion occurs in Child List***]

Else if type $=$ **2** , ***then*** :

SET LINK[***NEWPTR***] $:=$ ***CHILD***[***TEMPPTR***]

[*LINKING NEWNODE with the child node of Node in the main list ,where the target is present. [If any child node is linked with target node, then newNode will link to that child Node , that is linked with the target Node.*]]

SET CHILD*[*TEMPPTR*] ≔ *NEWPTR

[***The child section of the Target Node in the Main List will link to the New Node. This will make new child of the target node putting the new Node*** [***New Child***] ***before the older child of the target node.***]

Else:

Write: ``***Invalid Option***``

CALL FREE*(*NEWPTR*)** [As, the NewNode is not attached to main list neither child , hence free the memory allocated by NewNode pointer.***]

[***End of if structure***]

13. ***END Of FUNCTION***
